

DPCellSalaryWeb — Project Documentation

Compiled reference covering the salary calculation engine, institute approval workflow, DA Difference & Increment modules, frontend page rendering, master-screen save payloads, and live-DB diagnostics. Read the relevant section before touching that part of the system.

1. Salary Calculation Engine

Core function: `routes/salaryCalculate.js` → `calculateForEmployee(employeeId, asOfDate, basicPayOverride) + mapCalcToGridRow(calc)`. It resolves DA/HRA/MA/TA/CLA/NPS from the masters effective on `asOfDate` and recalculates gross/deductions/net from the Basic.

Rule: To recalculate salary for a changed Basic (e.g. an increment), pass `basicPayOverride` — never recompute components by hand.

- **Basic Pay** comes from `PayMatrixMaster` via `PayRevisionId + PayLevel + CellNo` (`resolveBasicPay`). An annual increment = the next `CellNo` in the same level.
- **DA rate history** already exists in `dbo.DAMaster` (`DAPercentage`, `EffectiveFrom`, `EffectiveTo`, `PayRevisionId`). A new `DARevisionMaster/DARevisionRates` pair would duplicate it — look up DA by effective date instead of creating a parallel table.
- `EmployeeMaster.MonthOfIncrement` (INT) already stores each employee's increment month — increments are **not** July-only anywhere in the schema.
- **Bill codes** live in `dbo.SalaryBillCodes`. `normalizeBillPayload` in `routes/salaryBillCodes.js` already generates difference codes: `BillCategory='Difference' + BillType='DA Difference'` → e.g. `AUG-2026-DA-DIFF`. A DA Difference bill reuses a `SalaryBillCodes` row — don't invent a second bill-code system.
- **Institute-wise approval workflow** is `dbo.SalaryBillInstituteWorkflow` (+ `SalaryBillApprovalHistory`), keyed (`SalaryBillCodeId`, `InstituteCode`), with helpers in `utils/salaryBillInstituteWorkflow.js`. Any new bill type keyed to a `SalaryBillCodeId` reuses this table for DRAFT / SUBMITTED / RETURNED / RESUBMITTED / VERIFIED / APPROVED / LOCKED / REJECTED.
- **Historical monthly snapshots** are `dbo.SalaryEmployeeDetails` rows joined to their `SalaryBillCodes` row by `SalaryYear + SalaryMonthNumber + BillCategory='Salary'`, scoped by `InstituteCode`. For any historical-period calculation, read the per-month snapshot — never reuse the current month's Basic across past months.
- `buildEmployeeRows` in `routes/salaryEntry.js` prefers an existing saved snapshot over recalculating — this is what keeps already-saved months immune to later master changes. Preserve that branch.
- **NPS rule:** `CEILING((TotalBasicPay + DA) * 10%, 1)` in `utils/salaryBasicCalc.js` — excludes HRA/MA/TA/CLA.

2. Master-Screen Save Payloads — the ...actor Spread Hazard

Every master screen builds its save payload as `{ ...formFields, ...actor }`, where `actor` carries the logged-in user for `CreatedBy/ModifiedBy`.

Rule: spread `actor` FIRST and the form fields LAST, and never give `actor` keys the same names as form fields.

Why this matters: In `UserMaster` the `actor` was `{userName, fullName}` — the same field names as the account being created — and being spread last it overwrote them. Creating user “ao” while logged in as “admin” actually POSTed `userName: "admin"`, so the backend’s correct duplicate-check returned “Username already exists.” The bug looked like a database or query fault and was neither.

How to apply: name `actor` keys `actorUserName / actorFullName` (the backend’s `actorFromBody` already reads them), and put `...actor` at the top of the object literal.

Corollary: for routes where a body field legitimately means the record (`users.js`), `actorFromBody` must NOT fall back to `body.userName / body.fullName`, or `CreatedBy` gets stamped with the new account’s own name.

`CLAMaster.jsx` and `RolePermissionMaster.jsx` use the same `...actor` spread but have no colliding field names, so they are unaffected — check for name collisions rather than assuming every master screen has this bug.

Diagnostic worth reusing: `GET /api/users/_diagnostics` reports the server, instance name, database and user count the RUNNING backend actually sees (no secrets) — distinguishes a code bug from the app talking to a different SQL Server instance than SSMS.

3. Frontend Page Rendering (AppShell)

There is **no react-router**. Navigation is page state in `components/AppShell.jsx`, synced to the URL hash by `readHashPage() / writeHashPage()` in `utils/accessControl.js`.

Registering a page needs THREE edits, or it silently renders nothing: 1. an import 2. a `renderAuthorized("<id>", <Component/>)` entry 3. an `id` in `modules.js`

...and the `GenericModule` fallback at the bottom of `AppShell` must exclude the `id`.

Two historical causes of a blank screen: - Cause 1:

`renderAuthorized` returned `null` when `canAccessPage` denied, and the `GenericModule` fallback excludes every registered `id` — so a denied page rendered nothing at all, with no message. It now returns an explicit “no access” notice. - **Cause 2:** page was validated only in the `useState` initializer, never again. When the user prop changed (session restored, re-login as another role), a now-unauthorized page `id` persisted and blanked the screen. A `useEffect` on `[user, page]` now re-validates and redirects.

`PageErrorBoundary` wraps every authorized page: a render exception used to unmount the whole tree into a white screen; it now shows the error and a “Back to Home” button.

`normalizeRole` maps “Super Admin” (the seeded `RoleId=1` name) to `ADMIN` via its `includes("SUPER")` branch, and `ADMIN` short-circuits `canAccessPage` to `true`. A blank master page for admin is a rendering bug, not a permission one.

Cheque Amount is `calculateChequeAmount({netSalary, incomeTax, professionalTax})` in `utils/salaryBasicCalc.js` = Net + IT + PT. Every consumer imports it — never recompute it inline, and never use Gross.

Manual salary fields (IncomeTax, ProfessionalTax, OtherDeduction, manual NPS) round-trip through `finalizeSnapshotAmounts` → `SalaryEmployeeDetails` → `mapSavedDetailToGridRow`. `preserveSavedNps = fromSnapshot` || `npsManual` is what stops a manually entered NPS from being recalculated away; `recalcFromBasic/basicDriven` deliberately force auto.

Offline suite: `npm run test:bill-month-acceptance` (74 assertions, scenarios A-J plus items 12 and 16).

4. DA Difference & Employee Increment Modules

Added 2026-09-01. Migrations: -
`backend/sql/schema/33_DADifferenceAndEmployeeIncrement.sql` — `npm run migrate:da-difference` -
`backend/sql/schema/34_DADifferenceNPSDeduction.sql` — `npm run migrate:da-difference-nps`

Tables: `DADifferenceBill`, `DADifferenceEmployeeDetails`, `DADifferenceMonthDetails`, `EmployeeIncrement`. **New columns:** `EmployeeMaster.IncrementDate`; `DADifferenceMonthDetails.NPSDeduction` / `NPSManual` / `NetDifferenceAmount`; `DADifferenceEmployeeDetails.TotalNPSDeduction` / `TotalNetDifferenceAmount`.

Rule: every additive schema change must use an existence guard. Migration 19 hit error 1913 (IX_SBIW_Bill_Status already existed) from an ungarded `CREATE INDEX`. Wrap `CREATE INDEX` in `NOT EXISTS (SELECT 1 FROM sys.indexes ...)` and columns in `COL_LENGTH(...)` IS NULL.

Reused, not duplicated: - `DAMaster` is the DA rate history. -
`SalaryBillCodes` holds the AUG-2026-DA-DIFF code. -
`SalaryBillInstituteWorkflow` + `SalaryBillApprovalHistory` carry the approval states.

Resolve DA rates from `DAMaster` by effective date; key any new bill-type workflow on (`SalaryBillCodeId`, `InstituteCode`).

DA Difference reads the per-month snapshot: `SalaryEmployeeDetails` joined to its `SalaryBillCodes` row on `SalaryYear` + `SalaryMonthNumber` + `BillCategory='Salary'`, filtered by `InstituteCode`. Never carry one month's Basic across the period.

Saved DA Difference amounts are read back from `DADifferenceMonthDetails`, never recomputed. Use `GET /api/da-difference/:id/detail` for saved bills; `/calculate` is preview only.

NPS on DA arrears defaults to `CEILING(DA Difference × 10%, 1)` and is user-editable. A row edited by hand carries `NPSManual=1` and its value is merged back into every later save and preview, so recalculation cannot revert it. NPS is the ONLY field accepted from the client, and only after validation (numeric, ≥ 0 , $\leq$ its own DA Difference); all amounts are recomputed server-side. Send `npsOverrides: [{employeeId, salaryYear, salaryMonthNumber, npsDeduction}]`; an invalid value aborts the whole save with 400.

Increments are applied in `buildEmployeeRows` (`routes/salaryEntry.js`) by passing the new Basic to `calculateForEmployee` as `basicPayOverride`, so every component recalculates. Records are written only on Save Draft / Submit, inside the same transaction as the salary snapshot — never on Get Data.

`EmployeeMaster` is never mutated by an increment; the current `cell/level/Basic` for a month is derived from `EmployeeIncrement` history via `resolveEmployeeStateForMonth`. A manual New Basic wins over the Pay Matrix cell, and later increments chain from it.

Offline suites (both stub the DB, no SQL Server needed): - `npm run test:da-difference` — 60 assertions, must stay 60/60. - `npm run test:acceptance` — 82 assertions covering scenarios A-O.

5. Bill Month vs Salary Month Isolation & Approval Workflow

A salary bill is identified by `BillMonth` + `SalaryMonth` + `SalaryYear` + `BillCategory` + `BillType` + `InstituteCode`, or by its exact `BillCode/SalaryBillCodeId`. **Never by SalaryMonth alone.**

Column formats — the biggest trap

`BillMonth` stores 'JUN-2026', but `SalaryMonth` stores the month **name**, 'June' (`salaryMonth: month.name` in `routes/salaryBillCodes.js`).

Why it matters: `WHERE/CASE BillMonth = SalaryMonth` never matches, so any “is this the canonical bill” test written that way silently never fires. It shipped twice — in the DA snapshot ordering and nearly in the bill-code ordering.

How to apply: detect the canonical bill by its CODE having no `-BM-XXX` suffix — `BillCode LIKE N'%-BM-%'` in SQL, `/-BM-[A-Z]{3}$/i` in JS — the same marker `resolveSalaryEntryBill` itself uses.

Default bill selection

`GET /api/salary-entry/bill-codes` orders `SalaryYear` DESC, `SalaryMonthNumber` DESC, `<canonical first>`, `BillCodeId` DESC, and `Salary Entry's` preference chain falls through to `list[0]`.

Why: with only `BillCodeId` DESC, the newest-created row won, so JUN-2026-BM-MAY (created after the master) became the default and `Salary Entry` opened on Bill Month MAY-2026. Never hard-code a month — the latest salary month comes from the ordering.

Bill / source-bill split

When `BillMonth` differs from `SalaryMonth`, `utils/resolveSalaryEntryBill.js` returns TWO rows: `bill` = the exact Bill-Month variant, `sourceBill` = the salary-month master. They are **independent bills**. Status gates must use the EXACT resolved bill via `statusGateBill(resolved)`, never `sourceBill`.

`ReturningBills` renders `<SalaryEntry>` directly as a child component and passes `initialBillCode` / `initialBillMonth` / `initialSalaryMonth` / `initialInstituteCode` / `returnedMode`. There is no hash or route hop and no `localStorage` in this path, so `AppShell` hash sync cannot lose the bill identity.

Status machine

Save Draft is **not** a transition. It used to set DRAFT unconditionally, which destroyed the RETURNED state and stranded bills outside both queues (“Returned Salary Bills: 0”). A draft save now keeps any AUDITOR_ACTIONABLE status (RETURNED, REJECTED); only an explicit Submit advances the bill.

The /returned listing predicate is Status = 'RETURNED' OR (Status = 'DRAFT' AND ReturnedToAuditorId IS NOT NULL) — the DRAFT clause recovers already-stranded rows. RESUBMITTED/SUBMITTED are excluded (back with the AO). upsertInstituteWorkflow sets ReturnedToAuditorId + AssignedAuditorId on RETURNED; RESUBMITTED preserves them.

Per-bill header fields

Per-(bill, institute) Salary Entry HEADER fields live on dbo.SalaryBillInstituteWorkflow, keyed (SalaryBillCodeId, InstituteCode): BillNo + BillDate (migration 37) and NPSScheduleNo (migration 45).

Rule: wire all FOUR backend points or the field silently fails to persist — the request read in saveEmployeesHandler, the UPDATE after upsertInstituteWorkflow, the getInstituteWorkflow read in GET /employees, and the bill:{} load response — then the frontend payload (Save Draft AND Submit) and the restore on load.

Roles & routing

/api/salary-bill-approval hosts BOTH the AO approval queue and the Auditor’s returned queue (GET /returned). The mount admits ACCOUNT_OFFICER + AUDITOR; every other route carries accountOfficerOnly explicitly. GET /returned derives role and auditor id from req.user, never the query string.

Auditor role permissions (migration 35) = everything except MASTER_USER_* and SALARY_APPROVAL*. A permission failure for an auditor is almost always a role-gate problem, not a missing permission.

DA Difference reads ONLY saved SalaryEmployeeDetails snapshots; “no snapshot” means that month was never saved for that institute — never an EmployeeMaster fallback. Ties break: canonical bill first (by the -BM- marker), then LOCKED/APPROVED/COMPLETED, then earliest BillCodeId; archived excluded.

Offline suites

Suite	Assertions
test:default-bill	47
test:returned-bills-listing	35
test:nps-schedule-no	42
test:bill-month-acceptance	75
test:returned-isolation	22
test:da-difference	61
test:acceptance	82
test:username	25

Suites that load the real db.js need the Windows-native msnodesqlv8 driver and **cannot run in the Linux bridge VM**: test:role-access, test:return-auditor, test:salary-approval-da, test:da-difference-

month-lock.

6. Live Database Diagnostics

Claude reaches the user's machine through a Linux VM. msnodesqlv8 is Windows-native and SQL Server uses Windows auth, so **Claude cannot query the live database directly**. Offline suites prove logic only — live schema and row state must be checked on the user's machine.

```
npm run diagnose:returned-bills
```

(backend/scripts/diagnoseReturnedBills.js) is **READ-ONLY** and answers, in one run: - DB_NAME/@SERVERNAME/instance actually used - existence of SalaryBillInstituteWorkflow and SalaryBillCodes, and that table's columns - which migrations are applied (m34, m37, m45) - all SalaryBillCodes rows for a salary year with their BillMonth - the workflow rows for an institute with Status/ReturnedToAuditorId/AssignedAuditorId/NPSScheduleNo - every auditor with how many returned bills each would see - the count of RETURNED rows with a NULL ReturnedToAuditorId

Takes optional args: institute code, bill code.

Common error decoder: - "Msg 208 Invalid object name 'dbo.SalaryBillInstituteWorkflow'" → the SSMS query window is on the wrong database, not that the table is missing (it's created by migration 29). - If switching to DPCELLSalaryWebDB then gives "Msg 207 Invalid column name 'NPSScheduleNo'" → the table exists, migration 45 simply has not been applied yet.

Month column formats — recurring trap:

SalaryBillCodes.SalaryMonth stores a month NAME ('June'); BillMonth stores a label ('JUN-2026') OR the same name, depending on which path wrote it (create without billMonth mirrors SalaryMonth). Never compare the two columns directly — use normalizeYearMonth + yearMonthKey from utils/salaryMonthKey.js, or detect a canonical bill by its code lacking a -BM-XXX suffix.

Salary Entry's displayed Bill Month for a normal open comes straight from the selected row's stored BillMonth (applyBillCodeMeta, preserveBillMonth=false). So "Bill Code JUN-2026 with Bill Month MAY-2026" is a **data value** on that row, not a display bug.

Open question — do not change without the user deciding: resolveBillMonth in routes/salaryBillCodes.js is documented as "BillMonth may differ from SalaryMonth for OLD salary (arrears) bills," so a canonical master code holding a foreign Bill Month may be intentional — even though the -BM- variant mechanism also exists for that purpose. There are two overlapping mechanisms; ask before constraining either.

Last compiled: 2026-09-23. Source: project memory notes for DPCellSalaryWeb (architecture, frontend rendering, DA Difference/Increment, bill-month workflow, actor-spread feedback, live-DB diagnostics).